HOMEWORK 1

16824 VISUAL LEARNING AND RECOGNITION (FALL 2025)
https://piazza.com/cmu/fall2025/16824/home

RELEASED: Wed, 17 Sep 2025
DUE: Wed, 1 Oct, 2025
Instructor: Jun-Yan Zhu
TAs: Ananya Bal, Zhixuan Liu, Eungyeup Kim, Jay Karhade

START HERE: Instructions

- **Collaboration policy:** All are encouraged to work together BUT you must do your own work (code and write up). If you work with someone, please include their name in your write-up and cite any code that has been discussed. If we find highly identical write-ups or code or lack of proper accreditation of collaborators, we will take action according to strict university policies. See the Academic Integrity Section detailed in the initial lecture for more information.
- **Late Submission Policy:** There are a **total of 5** late days across all homework submissions. Submissions that use additional late days will incur a 10% penalty per late day.
- **Submitting your work:**
  - We will be using Gradescope (`https://gradescope.com/`) to submit the Problem Sets. Please use the provided template only. You do **not** need any additional packages and using them is **strongly discouraged**. Submissions must be written in LaTeX. All submissions not adhering to the template will not be graded and receive a zero.
  - **Deliverables:** Please submit all the `.py` files. Add all relevant plots and text answers in the boxes provided in this file. To include plots you can simply modify the already provided latex code. Submit the compiled `.pdf` report as well.

NOTE: Partial points will be given for implementing parts of the homework even if you don't get the mentioned accuracy as long as you include partial results in this pdf.

1 PASCAL multi-label classification (20 points)

In this question, we will try to recognize objects in natural images from the PASCAL VOC dataset using a simple CNN.

- **Setup:** Run the command `bash q1_q2_classification/download_data.sh` to download the train and test splits. The images will be downloaded in `data/VOCdevkit/VOC2007/JPEGImages` and the corresponding annotations are in `data/VOCdevkit/VOC2007/Annotations`. `voc_dataset.py` contains code for loading the data. Fill in the method `preload_anno` in to preload annotations from XML files. Inside `__getitem__` add random augmentations to the image before returning it using [TORCHVISION.TRANSFORMS]. There are lots of options and experimentation is encouraged. Implement a suitable loss function inside `trainer.py` (you can pick one from here). Also, define the correct dimension in `simple_cnn.py`.
- **Question:** The file `train_q1.py` launches the training. Please choose the correct hyperparameters in lines 19-28. You should get a mAP of around 0.22 within 5 epochs.
- **Deliverables:**
  - The code should log values to a Tensorboard. Compare the `Loss/Train` and `mAP` curves of the model with and without data augmentations in the boxes below. You should include the two curves in a single plot for each metric.

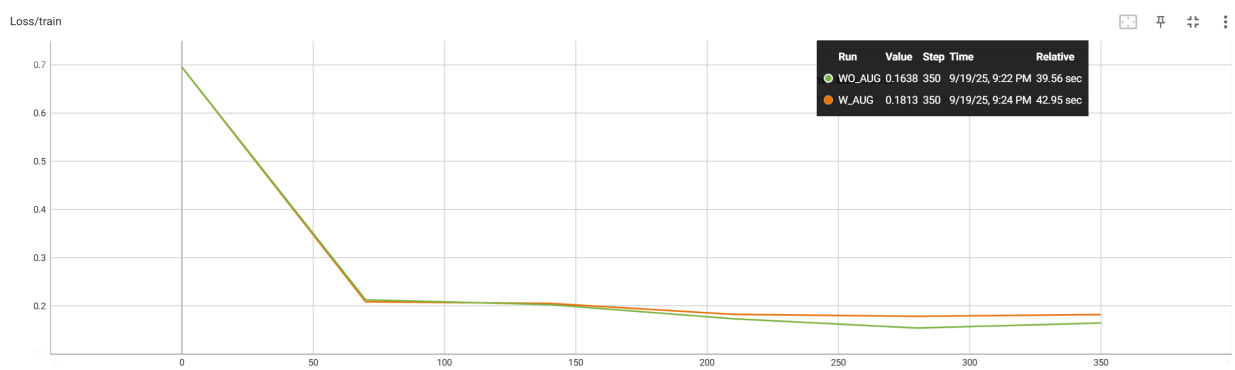

Figure 1.1: `Loss/Train` with and without data augmentations.

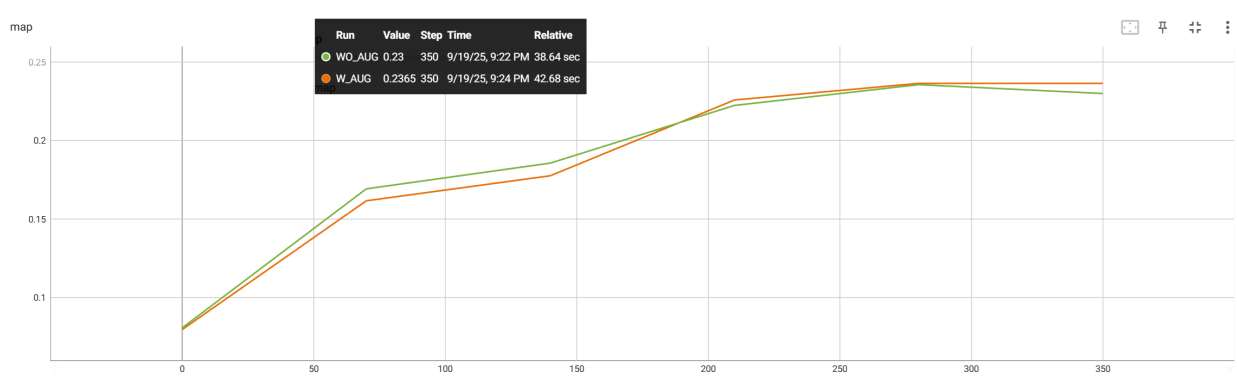

Figure 1.2: `mAP` with and without data augmentations.

– Report the `Loss/Train`, `mAP` and `learning_rate` curves of your best model logged to Tensorboard in the boxes below.

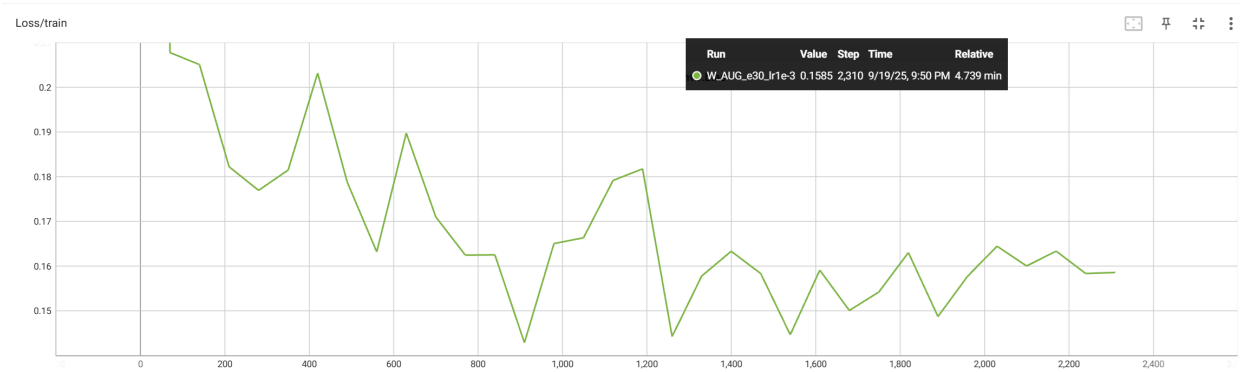

Figure 1.3: `Loss/Train` for simple CNN

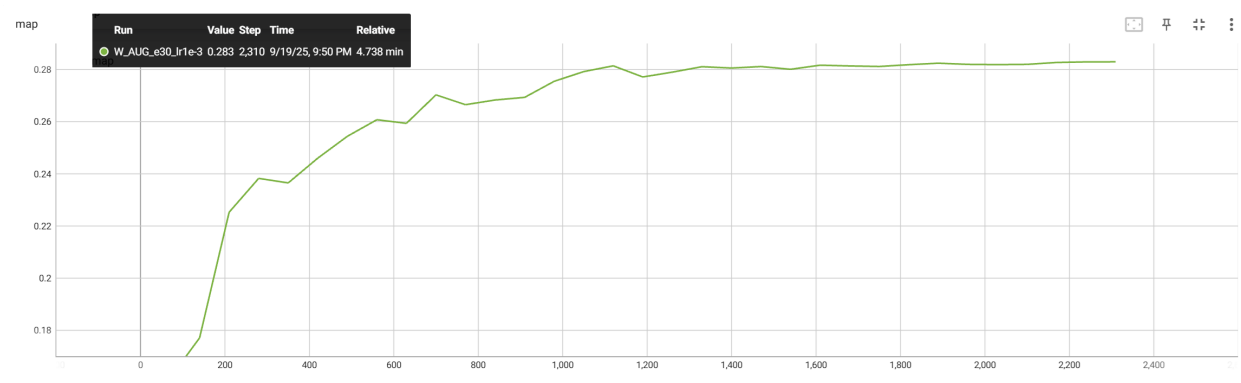

Figure 1.4: `mAP` for simple CNN

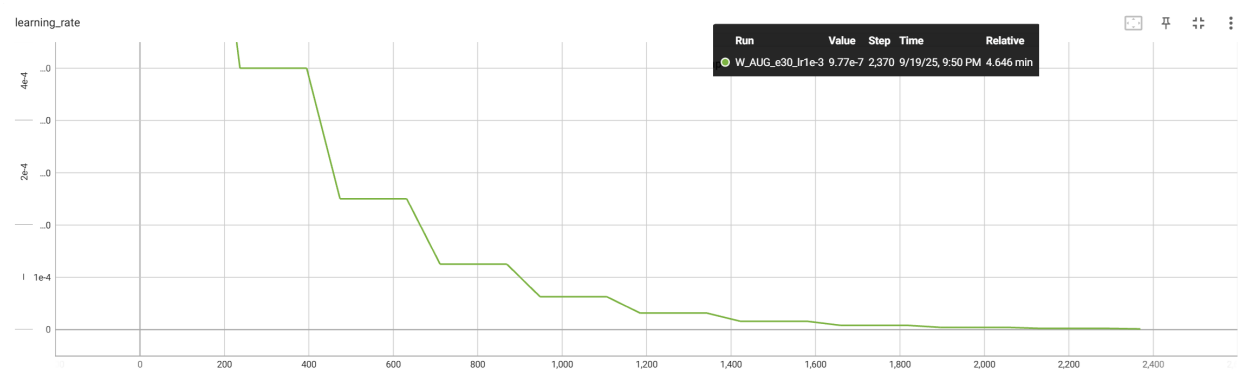

Figure 1.5: `learning_rate` for simple CNN

– Describe the hyperparameters you experimented with and the effects they had on the train and test metrics.

Solution:
Settings (Figure1.1–Figure1.2). I trained two models with identical hyperparameters to compare augmentation: `epochs=5`, `inp_size=64`, Adam `lr=1e-3`, `batch_size=64`, StepLR(`step_size=3`, `gamma=0.5`), `val_every=70`, `log_every=70`. Runs: `AUG=0` (no data augumentation) vs. `AUG=1`, (data augumentation on). **Outcome:** both losses decreased at a similar rate, but the augmented model achieved consistently higher test mAP, indicating better generalization.

Best model (Figure1.3–Figure1.5). I then extended training with the same optimizer/scheduler but `epochs=30` and augmentation enabled (`AUG=1`). The StepLR drops after epochs 3, 6, 9, ... stabilized training and improved performance, yielding a best test mAP of ≈ 0.283.

Conclusion. Data augmentation plus a longer schedule with LR decay gives higher mAP; short runs without augmentation fit the training set slightly faster but generalize worse.

2 Even deeper! Resnet18 for PASCAL classification (20 pts)

Hopefully, we get much better accuracy with the deeper models! Since 2012, much deeper architectures have been proposed. ResNet is one of the popular ones.

- **Setup:** Write a network module for the Resnet-18 architecture (refer to the original paper) inside `train_q2.py`. You can use Resnet-18 available in `torchvision.models` for this section. Use ImageNet pre-trained weights for all layers except the last one.
- **Deliverables:** Paste plots for the following in the box below.
  - Include curves of training loss, test MAP, learning rate, and histogram of gradients from Tensorboard for `layer1.1.conv1.weight` and `layer4.0.bn2.bias`.

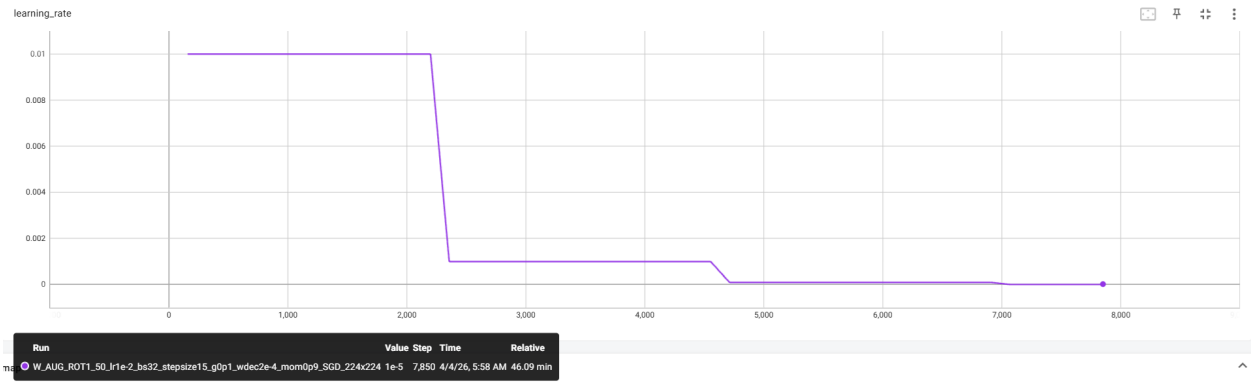

Figure 2.1: `learning_rate` for ResNet

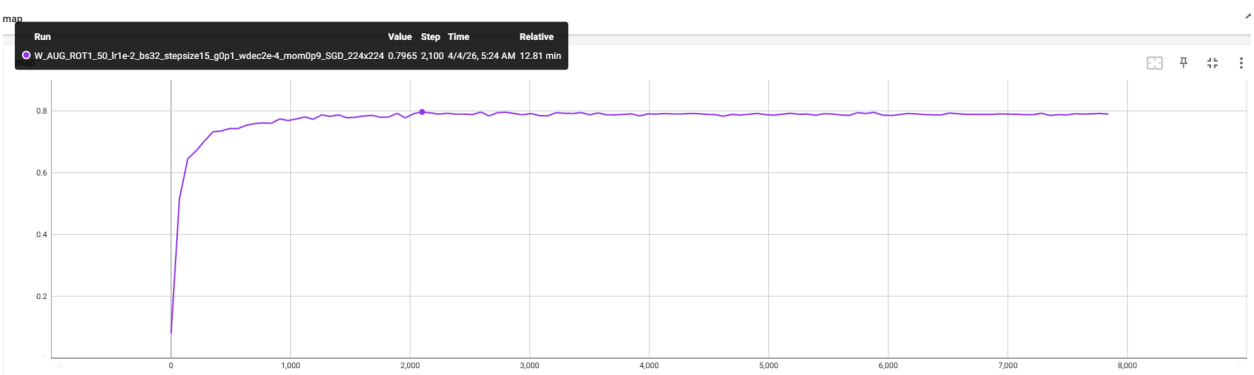

Figure 2.2: `mAP` for ResNet

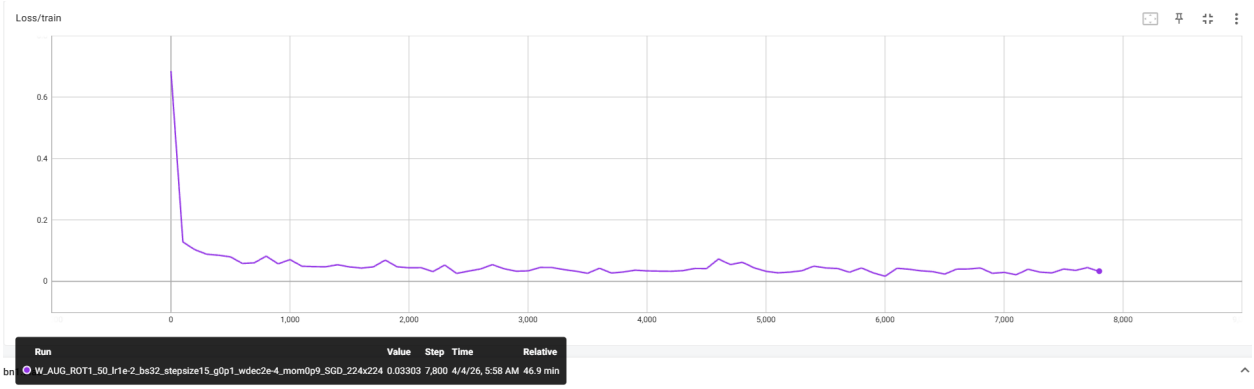

Figure 2.3: `Loss/Train` for ResNet

– How does the test mAP and training loss change over time? Why do you think this is happening?

Solution: The test mAP increases very quickly at the beginning, then improves more slowly, and finally plateaus around a high value (roughly 0.79) with only small fluctuations. The training loss shows the opposite trend: it drops sharply in the early stage, then continues decreasing more gradually, and eventually stabilizes at a low value.

This behavior is expected. Since ResNet-18 is initialized with ImageNet pre-trained weights, the model already starts with useful visual features, so it can learn the PASCAL classification task much faster than a network trained from scratch. That is why we see a rapid early decrease in training loss and a rapid increase in test mAP. After this initial phase, the model has already captured most of the easy transferable structure, so additional gains become smaller and both curves begin to flatten.

The learning rate schedule also helps explain the shape of the curves. When the learning rate is reduced, parameter updates become smaller, which usually makes training more stable and helps fine-tune the model near a local optimum. As a result, the training loss keeps decreasing slowly, while the test mAP improves only marginally and then saturates. The small oscillations are likely due to mini-batch SGD noise and the fact that mAP on the test set is a noisier evaluation metric than training loss.

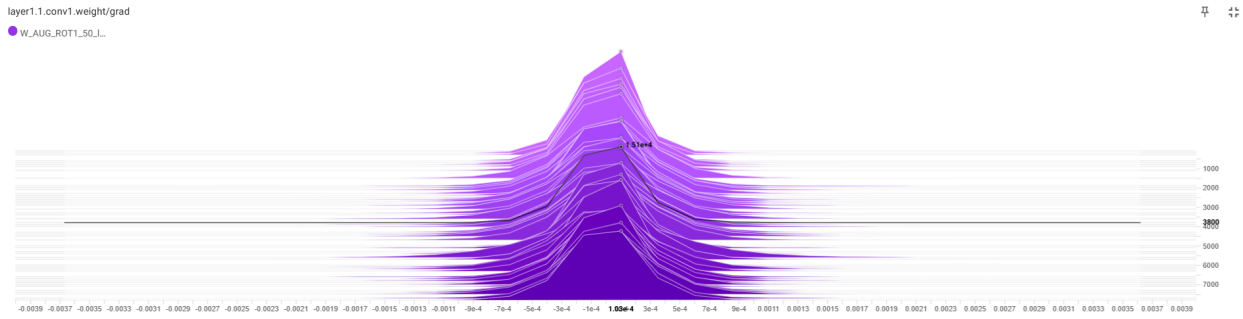

Figure 2.4: Histogram of `Conv1` layer

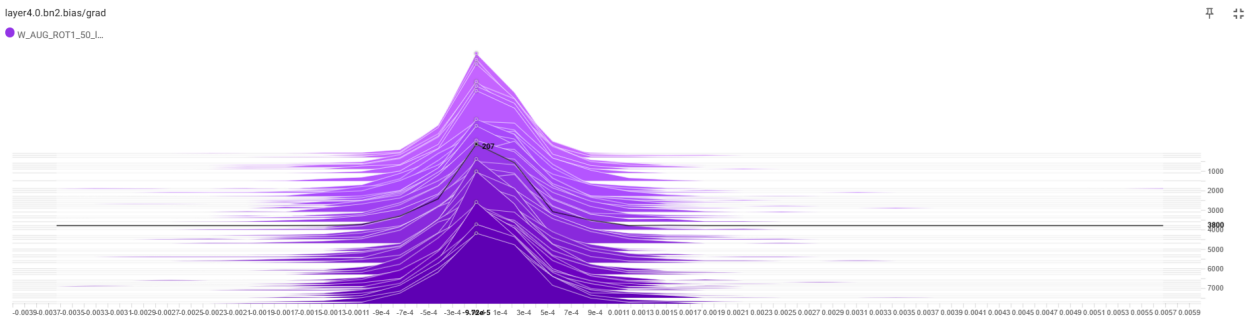

Figure 2.5: Histogram of `BN4` layer

– Compare the two histogram plots. How do they change over time? Why do you think this is happening?

Solution: Both gradient histograms are centered close to zero throughout training, but they differ in spread and evolution. The histogram for `layer1.1.conv1.weight` is relatively narrower and more symmetric, which suggests that the gradients in this early convolutional layer are small and stable. In contrast, the histogram for `layer4.0.bn2.bias` is broader and appears to have a slightly longer tail, meaning the gradients in this deeper layer are generally larger and more variable.

Over time, both histograms become more concentrated around zero. Early in training, the distributions are wider, indicating larger gradient updates. Later, they shrink and pile up closer to zero, showing that parameter updates are getting smaller as the model approaches convergence. This matches the behavior of the loss and mAP curves: once the network has learned most of the useful task-specific adjustments, fewer large updates are needed.

A likely reason for the difference between the two layers is that the early convolutional layer is initialized from ImageNet pretraining and already contains useful low-level features such as edges and textures, so it only needs modest fine-tuning. The deeper batch-normalization bias is much closer to the classifier and is more task-specific, so it receives stronger gradients in order to adapt the higher-level representation to PASCAL classes. In addition, the learning rate decay also helps explain why both histograms narrow over time: smaller learning rates lead to smaller effective updates and hence gradients that cluster more tightly around zero.

– We can also visualize how the feature representations specialize for different classes. Take 1000 random images from the test set of PASCAL, and extract ImageNet (finetuned) features from those images. Compute a 2D t-SNE (use sklearn) projection of the features, and plot them with each feature color-coded by the GT class of the corresponding image. If multiple objects are active in that image, compute the color as the "mean" color of the different classes active in that image. Add a legend explaining the mapping from color to object class.

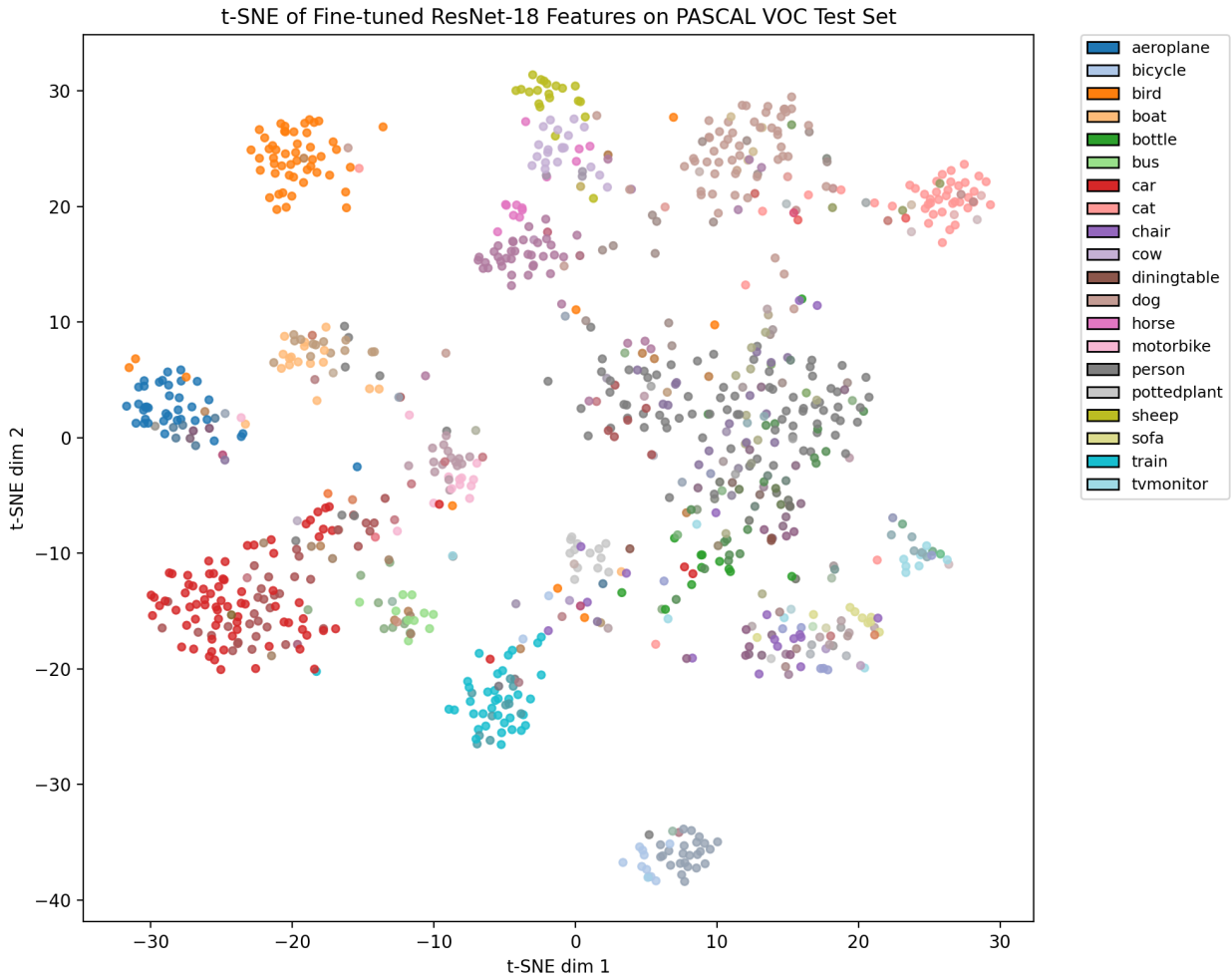

Figure 2.6: t-SNE

– Briefly describe what you observe in the t-SNE plot. Does this match your expectations?

Solution: From the t-SNE plot, several classes form relatively compact local clusters, which suggests that the fine-tuned ResNet-18 features preserve meaningful local neighborhood structure. In other words, images that are close to each other in the original high-dimensional feature space tend to stay close in the 2D visualization, so these local clusters are interpretable. At the same time, the central mixed region shows noticeable overlap across classes, which is expected because PASCAL VOC is a multi-label dataset and many images contain multiple objects.

However, the global arrangement of clusters should not be over-interpreted. Although some groups appear far apart or closer to one another in the 2D plot, t-SNE does not reliably preserve global distances. Therefore, it is reasonable to interpret local grouping and overlap, but not to make strong claims based on the absolute distances or relative positions between far-apart clusters.

This mostly matches my expectations. Since the network was fine-tuned on PASCAL VOC,

I would expect the learned features to contain class-specific structure, so some clustering is reasonable. However, I would not expect perfectly clean class boundaries because VOC is a multi-label dataset: many images contain multiple objects at once, so their features can lie between several semantic groups. In addition, t-SNE mainly preserves local neighborhoods rather than global distances, so partial clustering with some overlap is a realistic and reasonable outcome.

3 Supervised Object Detection: FCOS (60 points)

In this problem, we'll be implementing supervised Fully Convolutional One-stage Object Detection (FCOS).

- **Setup**. This question will require you to implement several functions in `detection_utils.py` and `one_stage_detector.py` in the `detection` folder. Instructions for what code you need to write are in the `README` in the `detection` folder of the assignment.

  We have also provided a testing suite in `test_object_detection.py`. First, run the test suite and ensure that all the tests are either skipped or passed. Make sure that the Tensorboard visualization works by running 'python3 train.py –visualize_gt'; this should upload some examples of the training data with bounding boxes to Tensorboard. Make sure everything is set up properly before moving on.

  Then, run the following to install the `mAP` computation software we will be using.

```
cd <path/to/hw/>/detection
pip install wget
rm -rf mAP
git clone https://github.com/Cartucho/mAP.git
rm -rf mAP/input/*
```

  Next, open `detection/one_stage_detector.py` and `detection/detection_utils.py`. At the top of the files are detailed instructions for where and what code you need to write. Follow the `README` and all the instructions for implementation.

- **Deliverables.**
  - It's always a good idea to check if your model can overfit on a small subset of the data, otherwise there may be some major bugs in the code. Train your FCOS model on a small subset of the training data and make sure the model can overfit. Include the loss curve from over-fitting below.

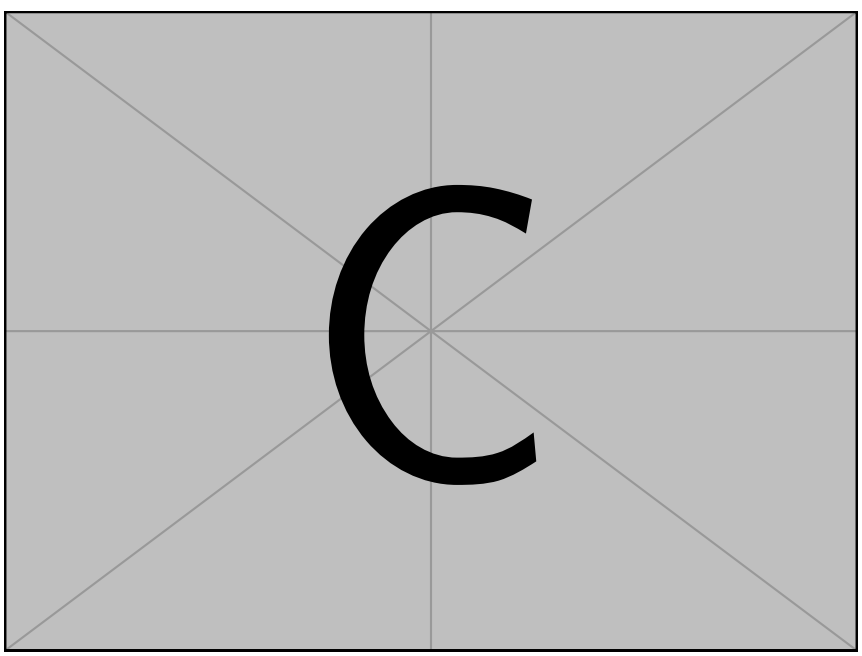

Figure 3.1: Overfitting Training Curve

– Next, train FCOS on the full training set and include the loss curve below.

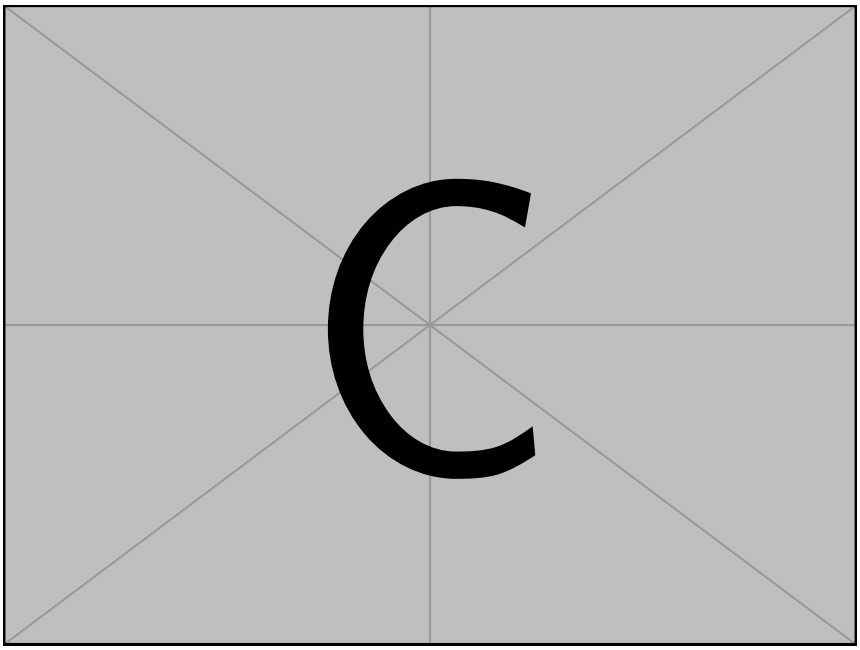

Figure 3.2: Full Training Curve

– Include the plot of the model's class-wise and average mAP. If everything is correct, your implementation should reach a mAP of at least 20.

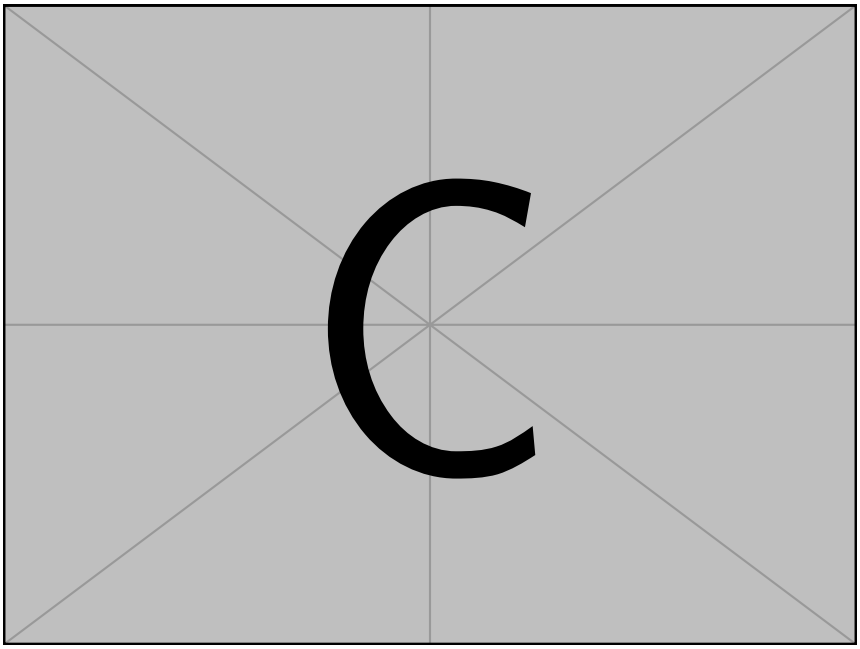

Figure 3.3: mAP plots

– Paste a screenshot of the Tensorboard visualizations of your model inference results from run-

ning inference with the `--test_inference` flag on.

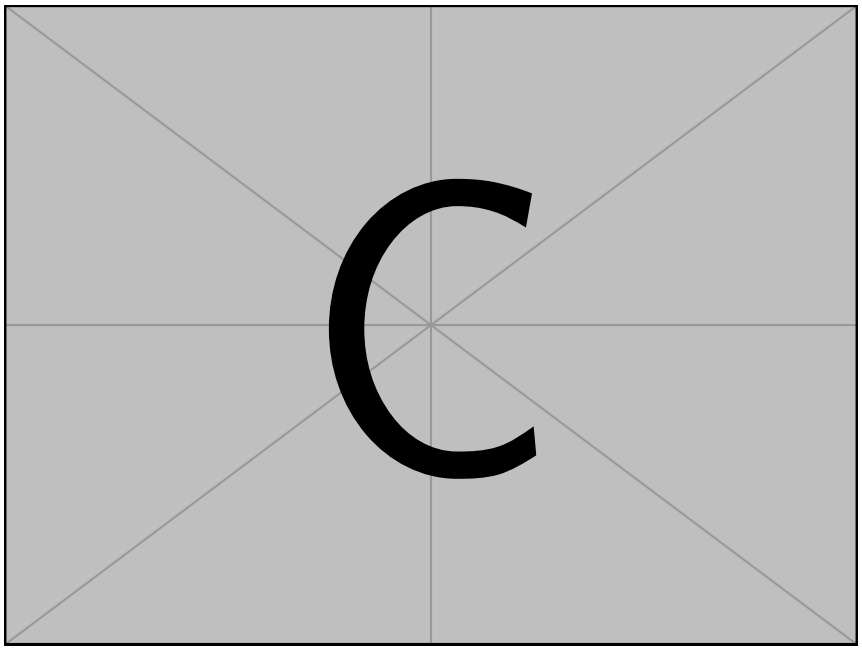

Figure 3.4: Tensorboard Inference Visualization

- What can you conclude from the above visualizations? When does the model succeed or fail? How can you improve the results for the failure cases?

Solution:

Collaboration Survey Please answer the following:

1. Did you receive any help whatsoever from anyone in solving this assignment?

   ◯ Yes

   ◯ No

   - If you answered 'Yes', give full details:
   - (e.g. "Jane Doe explained to me what is asked in Question 3.4")

2. Did you give any help whatsoever to anyone in solving this assignment?

   ◯ Yes

   ◯ No

   - If you answered 'Yes', give full details:
   - (e.g. "I pointed Joe Smith to section 2.3 since he didn't know how to proceed with Question 2")

3. Note that copying code or writeup even from a collaborator or anywhere on the internet violates the Academic Integrity Code of Conduct.